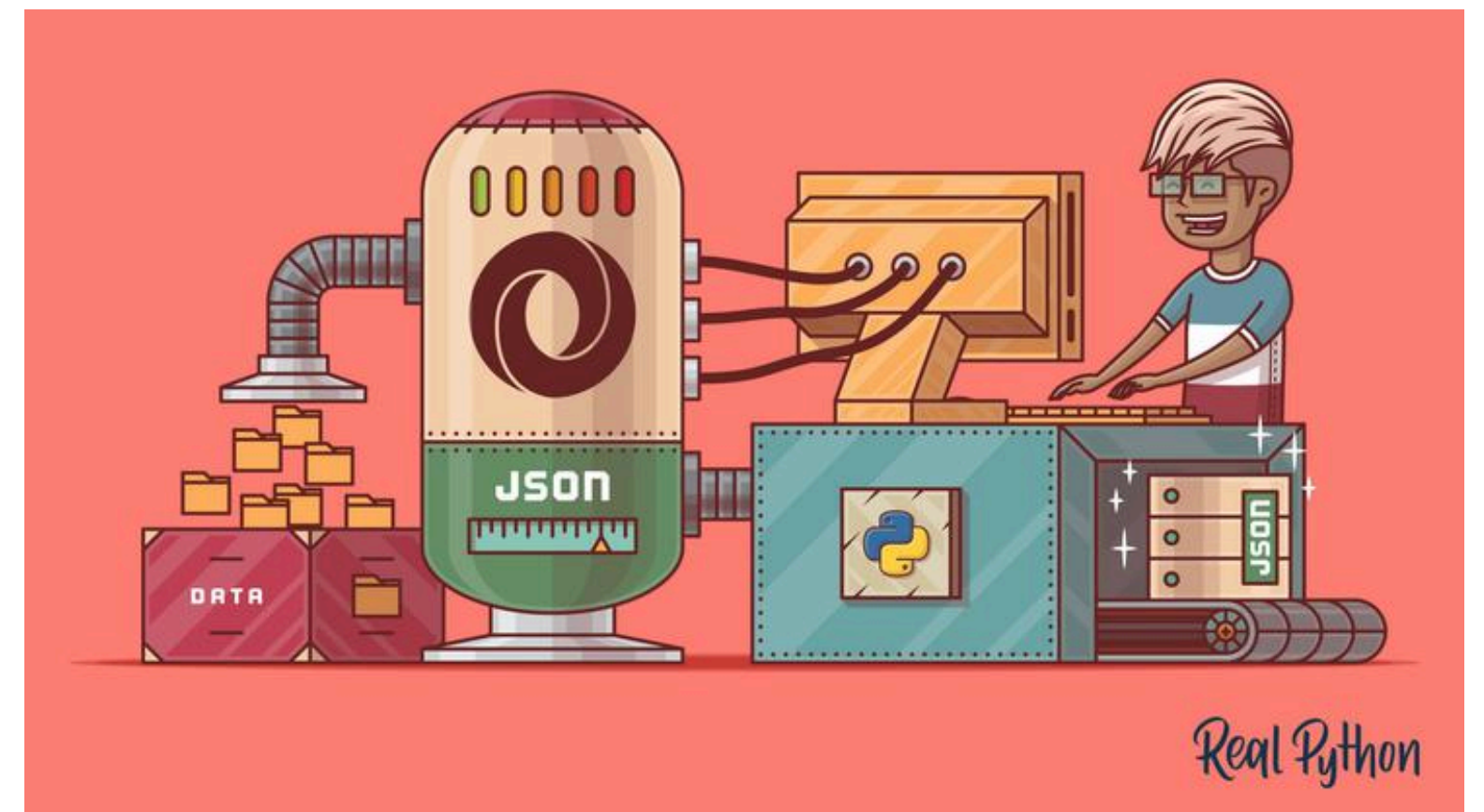


Модуль json



Повторим что такое JSON?

JSON часто применяют, когда разрабатывают API и веб-приложения.

Основной принцип работы интернета — обмен данными. Они бывают разных видов, например файл, строка или число. Есть структура данных, которая позволяет быстро и просто воссоздавать объекты и обмениваться этими данными по сети, — JSON.

JSON (англ. JavaScript Object Notation) — текстовый формат обмена данными, основанный на JavaScript. Как и многие другие текстовые форматы, JSON легко читается людьми. Формат JSON был разработан Дугласом Крокфордом.

JSON-текст представляет собой (в закодированном виде) одну из двух структур:

- Набор пар ключ: значение. В различных языках это реализовано как запись, структура, словарь, хеш-таблица, список с ключом или ассоциативный массив. Ключом может быть только строка (регистрозависимость не регулируется стандартом, это остаётся на усмотрение программного обеспечения. Как правило, регистр учитывается программами — имена с буквами в разных регистрах считаются разными, например), значением — любая форма. Повторяющиеся имена ключей допустимы, но не рекомендуются стандартом; обработка таких ситуаций происходит на усмотрение программного обеспечения, возможные варианты — учитывать только первый такой ключ, учитывать только последний такой ключ, генерировать ошибку.
- Упорядоченный набор значений. Во многих языках это реализовано как массив, вектор, список или последовательность.

Повторим что такое JSON?

Структуры данных, используемые JSON, поддерживаются любым современным языком программирования, что и позволяет применять JSON для обмена данными между различными языками программирования и программными системами.

В качестве значений в JSON могут быть использованы:

- **запись** — это неупорядоченное множество пар **ключ:значение**, заключённое в фигурные скобки «{ }». Ключ описывается **строкой**, между ним и значением стоит символ «:». Пары ключ-значение отделяются друг от друга запятыми.
- **массив** (одномерный) — это упорядоченное множество **значений**. Массив заключается в квадратные скобки «[]». Значения разделяются запятыми. Массив может быть пустым, то есть не содержать ни одного значения. Значения в пределах одного массива могут иметь разный тип.
- **число** (целое или вещественное).
- **литералы** true (логическое значение «истина»), false (логическое значение «ложь») и null.
- **строка** — это упорядоченное множество из нуля или более символов юникода, заключённое в двойные кавычки. Символы могут быть указаны с использованием escape-последовательностей, начинающихся с обратной косой черты «\» (поддерживаются варианты \", \\, \/, \t, \n, \r, \f и \b), или записаны шестнадцатеричным кодом в кодировке Unicode в виде \uFFFF.

Сериализация и десериализация

В Python есть множество библиотек, чтобы работать с JSON, но мы рассмотрим встроенную библиотеку JSON Python. Она позволяет приводить любые структуры данных к JSON-объекту — вплоть до пользовательских классов. А из него получать совместимую для работы в Python сущность — объект языка.

Как вы уже знаете упаковка объектов в байтовую последовательность называется сериализацией. А распаковка байтов в объекты языка программирования, приведение последовательности назад к типам и структурам, — десериализацией.

В байты данные необходимо переводить, чтобы отправлять их по сети или локально другому приложению, так как иной формат передать невозможно. Вот так преобразовывают данные из объектов Python в JSON и обратно:

А вот результат выполнения нашего кода:

```
{"my_string": "some_string", "my_integer": 123, "my_list": [1, 2, 3, 4, 5], "my_bool": true, "my_none": null}
<class 'str'>
{'my_string': 'some_string', 'my_integer': 123, 'my_list': [1, 2, 3, 4, 5], 'my_bool': True, 'my_none': None}
<class 'dict'>
```

```
import json

my_dict = {
    'my_string': 'some_string',
    'my_integer': 123,
    'my_list': [1, 2, 3, 4, 5],
    'my_bool': True,
    'my_none': None
}

my_json = json.dumps(my_dict)
print(my_json)
print(type(my_json))

my_dict = json.loads(my_json)
print(my_dict)
print(type(my_dict))
```


Особенности работы с JSON

Чтение

Для чтения в модуле json есть два метода:

- **json.load** - метод считывает **файл** в формате JSON и возвращает объекты Python
- **json.loads** - метод считывает **строку** в формате JSON и возвращает объекты Python

Запись

Запись файла в формате JSON также осуществляется достаточно легко.

Для записи информации в формате JSON в модуле json также два метода:

- **json.dump** - метод записывает **объект** Python в файл в формате JSON
- **json.dumps** - метод возвращает строку в формате JSON

Изменение типа данных

Еще один важный аспект преобразования данных в формат JSON: данные не всегда будут того же типа, что исходные данные в Python.

Например, кортежи при записи в JSON превращаются в списки.

Так происходит из-за того, что в JSON используются другие типы данных и не для всех типов данных Python есть соответствия.

Таблица конвертации данных Python в JSON:

Python	JSON
dict	object
list, tuple	array
str	string
int, float	number
True	true
False	false
None	null

и наоборот

JSON	Python
object	dict
array	list
string	str
number (int)	int
number (real)	float
true	True
false	False
null	None

Ограничение по типам данных

В формат JSON нельзя записать словарь, у которого ключи - кортежи. Кроме того, в JSON ключами словаря могут быть только строки. Но, если в словаре Python использовались числа, ошибки не будет. Вместо этого выполнится конвертация чисел в строки.

Особенности работы с JSON

Рассмотрим параметры которые можно дополнительно передавать в метода чтения и записи
`json.load(fp, cls=None, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, object_pairs_hook=None, **kw)`

- `fp` - файловый поток;
- `cls=None` - пользовательский подкласс `JSONDecoder`;
- `object_hook=None` - пользовательская функция для преобразования каждого литерала словаря;
- `parse_float=None` - пользовательская функция для преобразования литералов, похожих на `float`;
- `parse_int=None` - пользовательская функция для преобразования литералов, похожих на `int`;
- `parse_constant=None` - пользовательская функция для преобразования литералов `-Infinity`, `Infinity` и `NaN`;
- `object_pairs_hook=None` - пользовательская функция для преобразования литералов, декодированных упорядоченным списком пар.

Возвращаемое значение: объект Python.

`json.loads(s, *, encoding=None, cls=None, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, object_pairs_hook=None, **kw)`

- `s` - строка в формате JSON;
- `cls=None` - пользовательский подкласс `JSONDecoder`;
- `object_hook=None` - пользовательская функция для преобразования каждого литерала словаря;
- `parse_float=None` - пользовательская функция для преобразования литералов, похожих на `float`;
- `parse_int=None` - пользовательская функция для преобразования литералов, похожих на `int`;
- `parse_constant=None` - пользовательская функция для преобразования литералов `-Infinity`, `Infinity` и `NaN`;
- `object_pairs_hook=None` - пользовательская функция для преобразования литералов, декодированных упорядоченным списком пар.

Возвращаемое значение: объект Python.

Особенности работы с JSON

Рассмотрим параметры которые можно дополнительно передавать в метода чтения и записи `json.dump(obj, fp, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, cls=None, indent=None, separators=None, default=None, sort_keys=False, **kw)`

- `obj` - объект Python;
- `fp` - поток в формате JSON;
- `skipkeys=False` - игнорирование неизвестных типов ключей в словарях;
- `ensure_ascii=True` - экранирование не-ASCII символов;
- `check_circular=True` - проверка циклических ссылок;
- `allow_nan=True` - представление значений `nan`, `inf`, `-inf` в JSON;
- `cls=None` - метод, для сериализации дополнительных типов;
- `indent=None` - количество отступов при сериализации;
- `separators=None` - разделители используемые в JSON;
- `default=None` - пользовательская функция для объектов, которые не могут быть сериализованы;
- `sort_keys=False` - сортировка словарей на выходе.

Возвращаемое значение: поток в формате JSON, который записывается методом `fp.write()`.

Особенности работы с JSON

Рассмотрим параметры которые можно дополнительно передавать в метода чтения и записи `json.dumps(obj, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, cls=None, indent=None, separators=None, default=None, sort_keys=False, **kw)`

- `obj` - объект Python;
- `skipkeys=False` - игнорирование неизвестных типов ключей в словарях;
- `ensure_ascii=True` - экранирование не-ASCII символов;
- `check_circular=True` - проверка циклических ссылок;
- `allow_nan=True` - представление значений `nan`, `inf`, `-inf` в JSON;
- `cls=None` - метод, для сериализации дополнительных типов;
- `indent=None` - количество отступов при сериализации;
- `separators=None` - разделители используемые в JSON;
- `default=None` - функция для объектов, которые не могут быть сериализованы;
- `sort_keys=False` - сортировка словарей.

Возвращаемое значение: строку `str` формата JSON.

Более подробно все это можно рассмотреть по ссылке:

<https://docs-python.ru/standart-library/modul-json-python/priemy-raboty-modulem-json/>

Особенности работы с JSON

Dumps позволяет создать JSON-строку из переданного в нее объекта. Loads — преобразовать строку назад в объекты языка.

Dump и load используют, чтобы сохранить результат в файл или воссоздать объект. Работают они схожим образом, но требуют передачи специального объекта для работы с файлом — filehandler.

```
import json

my_dict = {
    'my_string': 'some_string',
    'my_integer': 123,
    'my_list': [1, 2, 3, 4, 5],
    'my_bool': True,
    'my_none': None
}

with open('my_data.json', 'w', encoding='utf-8') as fh:
    json.dump(my_dict, fh, ensure_ascii=False, indent=1)

with open('my_data.json', 'r') as fh:
    python_data = json.load(fh)

print(python_data)
```

V V V

```
{'my_string': 'some_string', 'my_integer': 123, 'my_list': [1, 2, 3, 4, 5], 'my_bool': True, 'my_none': None}
```



Как работать с пользовательскими объектами

Пользовательские классы не относятся к JSON-сериализуемым. Это значит, что просто применить к ним функции `dumps`, `loads` или `dump` и `load` не получится:

```
import json

class MyCat:
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

my_cat = MyCat("Франц", 3, "самец")
json_data = json.dumps(my_cat)
```

>>> File "C:\Users\suz17\AppData\Local\Programs\Python\Python312\Lib\json\encoder.py", line 180, in default
raise TypeError(f'Object of type {o.__class__.__name__} '
TypeError: Object of type MyCat is not JSON serializable

```
import json

class MyCat:
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

my_cat = MyCat("Франц", 3, "самец")
with open('my_cat.json', 'w', encoding='utf-8') as fh:
    json.dump(my_cat, fh)
```

>>> File "C:\Users\suz17\AppData\Local\Programs\Python\Python312\Lib\json\encoder.py", line 180, in default
raise TypeError(f'Object of type {o.__class__.__name__} '
TypeError: Object of type MyCat is not JSON serializable

Как видим мы в обоих случаях получаем одну и ту же ошибку. Объект типа `MyCat` не сериализуется JSON



Как работать с пользовательскими объектами

Но это не значит что мы ничего не сможем сделать. Я предложу вам 3 варианта решения данной проблемы. Это не так удобно и просто как при помощи `pickle`. Но это не значит что ничего нельзя сделать.

1й вариант - написать функцию.

Чтобы сериализовать пользовательский объект в JSON-структуру данных, нужен аргумент `default`.

Указывайте вызываемый объект, то есть функцию или статический метод.

Чтобы получить аргументы класса с их значениями, нужна встроенная функция `__dict__`, потому что любой класс — это словарь со ссылками на значения по ключу.

Чтобы сериализовать атрибуты класса и их значения в JSON, напомним функцию (будем использовать как лямбда функцию, так и обычную но в форме `@staticmethod`):



Как работать с пользовательскими объектами функция

```
import json

class MyCat:
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

my_cat = MyCat("Франц", 3, "самец")
json_data = json.dumps(my_cat,
                        default=lambda my_cat: my_cat.__dict__,
                        ensure_ascii=False, indent=2)

print(json_data)
python_data = json.loads(json_data)
print(python_data)

with open('my_cat.json', 'w', encoding='utf-8') as fh:
    json.dump(my_cat, fh,
              default=lambda my_cat: my_cat.__dict__,
              ensure_ascii=False, indent=2)

with open('my_cat.json', 'r', encoding='utf-8') as fh:
    python_data_from_file = json.load(fh)
print(python_data_from_file)
```

Вот результат работы в терминале

```
{
  "name": "Франц",
  "age": 3,
  "gender": "самец"
}
>>> {'name': 'Франц', 'age': 3, 'gender': 'самец'}
{'name': 'Франц', 'age': 3, 'gender': 'самец'}
```

А вот содержимое файла my_cat.json

```
{
  "name": "Франц",
  "age": 3,
  "gender": "самец"
}
>>>
```

Как видим информация об объекте файла записана успешно. Она есть как в терминале так и в файле



Как работать с пользовательскими объектами функция

```
import json

class MyCat:
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

    @staticmethod
    def to_json(obj):
        if isinstance(obj, MyCat):
            result = obj.__dict__
            result['className'] = obj.__class__.__name__
            return result

my_cat = MyCat("Франц", 3, "самец")
json_data = json.dumps(my_cat, default=MyCat.to_json,
                       ensure_ascii=False, indent=2)

print(json_data)
python_data = json.loads(json_data)
print(python_data)

with open('my_cat_1.json', 'w', encoding='utf-8') as fh:
    json.dump(my_cat, fh, default=MyCat.to_json,
              ensure_ascii=False, indent=2)

with open('my_cat_1.json', 'r', encoding='utf-8') as fh:
    python_data_from_file = json.load(fh)
print(python_data_from_file)
```

Вот результат работы в терминале

```
{
  "name": "Франц",
  "age": 3,
  "gender": "самец"
}
>>> {'name': 'Франц', 'age': 3, 'gender': 'самец'}
{'name': 'Франц', 'age': 3, 'gender': 'самец'}
```

А вот содержимое файла my_cat_1.json

```
>>> {
  "name": "Франц",
  "age": 3,
  "gender": "самец"
}
```

В данном случае мы использовали статический метод для записи информации об экземпляре класса класса, на мой взгляд этот метод более понятен чем использование лямбда-функции, тем более мы можем записать таким образом все что нам необходимо. В данном случае Мы добавили название класса в получаемую структуру. Такой подход позволяет безошибочно понять: сущность какого класса нужно десериализовать в объект.



Как работать с пользовательскими объектами класс-кодер

```
import json

class MyCat:
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

    def get_data(self):
        return self.name, self.age, self.gender

class MyCatEncoder(json.JSONEncoder):
    def default(self, o):
        return {'Name': o.name,
                'Age': o.age,
                'Gender': o.gender,
                'Get_name': o.get_data(),
                'ClassName': o.__class__.__name__}

my_cat = MyCat("Франц", 3, "самец")
json_data = json.dumps(my_cat, cls=MyCatEncoder, ensure_ascii=False, indent=2)
print(json_data)
python_data = json.loads(json_data)
print(python_data)

with open('my_cat_encode.json', 'w', encoding='utf-8') as fh:
    json.dump(my_cat, fh, cls=MyCatEncoder,
              ensure_ascii=False, indent=2)

with open('my_cat_encode.json', 'r', encoding='utf-8') as fh:
    python_data_from_file = json.load(fh)
print(python_data_from_file)
```

Вот результат работы в терминале

```
>>> {
  "Name": "Франц",
  "Age": 3,
  "Gender": "самец",
  "Get_name": [
    "Франц",
    3,
    "самец"
  ],
  "ClassName": "MyCat"
}
{'Name': 'Франц', 'Age': 3, 'Gender': 'самец', 'Get_name': ['Франц', 3, 'самец'], 'ClassName': 'MyCat'}
{'Name': 'Франц', 'Age': 3, 'Gender': 'самец', 'Get_name': ['Франц', 3, 'самец'], 'ClassName': 'MyCat'}
```

А вот содержимое файла my_cat_encode.json

```
>>> {
  "Name": "Франц",
  "Age": 3,
  "Gender": "самец",
  "Get_name": [
    "Франц",
    3,
    "самец"
  ],
  "ClassName": "MyCat"
}
```

Здесь мы используем класс - расширитель для json. Он позволяет нам отобрать нужные атрибуты и методы, о которых мы хотим добавить информацию в наш файл, но обратите внимание, мы вызываем атрибут а не передаем его адрес в памяти. Так мы получим результат его работы, иногда этого может быть достаточно чтобы понять принцип работы метода.

Как работать с пользовательскими объектами класс-адаптер

Идея в том, чтобы написать класс, который приводит к JSON пользовательские объекты и восстанавливает их. Определим класс фигуры, формы и цвета:

```
import json

class Figure:
    def __init__(self, title, form, color):
        self.title = title
        self.form = form
        self.color = color

    def __str__(self):
        return f"Figure: {self.title}, {repr(self.form)}, {repr(self.color)}"

class Form:
    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return f"<Form: {self.name}>"

class Color:
    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return f"<Color: {self.name}>"
```

Далее добавляем класс-адаптер, который принимает данные и адаптирует их в json и обратно:

```
class JSONDataAdapter:
    @staticmethod
    def to_json(o):
        if isinstance(o, Figure):
            return json.dumps({
                "title": o.title,
                "form": o.form.name,
                "color": o.color.name,
            })

    @staticmethod
    def from_json(o):
        o = json.loads(o)

        try:
            form = Form(o["form"])
            color = Color(o["color"])
            figure = Figure(o["title"], form, color)
            return figure
        except AttributeError:
            print("Неверная структура")
```

Как работать с пользовательскими объектами класс-адаптер

Определяем объекты и наделяем их необходимыми атрибутами, и запускаем программу.

```
if __name__ == '__main__':  
    # создадим несколько цветов  
    black = Color("Black")  
    yellow = Color("Yellow")  
    green = Color("Green")  
  
    # несколько форм  
    round = Form("Rounded")  
    square = Form("Squared")  
  
    # объекты фигур  
    figure_one = Figure("Black Square", form=square, color=black)  
    figure_two = Figure("Yellow Circle", form=round, color=yellow)  
  
    print("Отображение объектов")  
    print(figure_one)  
    print(figure_two)  
    print()  
  
    # преобразуем данные в JSON  
    jone = JSONDataAdapter.to_json(figure_one)  
    jtwo = JSONDataAdapter.to_json(figure_two)  
  
    print("Отображение Json")  
    print(jone)  
    print(jtwo)  
    print()  
  
    # восстановим объекты  
    restored_one = JSONDataAdapter.from_json(jone)  
    restored_two = JSONDataAdapter.from_json(jtwo)
```

Отображение объектов

Figure: Black Square, <Form: Squared>, <Color: Black>

Figure: Yellow Circle, <Form: Rounded>, <Color: Yellow>

Отображение Json

>>> {"title": "Black Square", "form": "Squared", "color": "Black"}

{"title": "Yellow Circle", "form": "Rounded", "color": "Yellow"}

Отображение восстановленных объектов

Figure: Black Square, <Form: Squared>, <Color: Black>

Figure: Yellow Circle, <Form: Rounded>, <Color: Yellow>